

Executive Summary: Dupire vs. Neural-Network Delta Hedging

Oscar Arevalo, Micheal O Cobhthaigh, Alec Hewitt, Ziqing Zhang

July 9, 2026

Project Objective

This project compares two approaches to delta hedging a short SPY call option: a classical Dupire local-volatility approach and a neural-network-based hedge ratio model. The goal is not to build a live trading system, but to evaluate how two different modeling frameworks translate market data into hedge ratios and how those hedge ratios behave in an out-of-sample daily rebalancing simulation.

The central question is:

How well can a classical local-volatility model and a data-driven neural network control the risk of a short option position through dynamic stock hedging?

Data Pipeline

The data construction is handled in `gen_dat.ipynb`. The notebook reads SPY option archive files from `archive/*.parquet`, cleans the raw quote columns, and converts the option records into a project-standard schema. The analysis uses call options with fields such as quote date, expiration, strike, bid, ask, last price, implied volatility, volume, and option Greeks from the archive.

The generated datasets are then consumed by `dupire_vs_nn_hedging.ipynb`. That notebook requires `data/historical_options.csv`; if this file is not available, the notebook stops rather than silently using synthetic option data. The current run uses real historical option data generated from the SPY archive.

The loader applies consistent cleaning:

- restricts the study to call options;
- computes mid price as $(\text{bid} + \text{ask})/2$;
- computes time to maturity T in years;
- computes moneyness K/S and log-moneyness $\log(K/S)$;
- filters to positive bid/ask quotes, positive open interest, positive implied volatility, maturities between 7 and 365 days, and moneyness between 0.8 and 1.2.

The stock data are SPY closing prices with daily returns, log returns, and 20-day and 60-day realized volatility estimates. These stock features are merged onto the option observations by date.

Financial Motivation

The Black–Scholes model assumes constant volatility:

$$dS_t = rS_t dt + \sigma S_t dW_t.$$

In observed option markets, however, the volatility implied by option prices varies across strike and maturity:

$$\sigma_{\text{imp}} = \sigma_{\text{imp}}(K, T).$$

This implied volatility surface shows that one constant volatility parameter is too restrictive for fitting the cross-section of traded option prices.

The presentation in `dupire_presentation/` motivates the Dupire model from this observation. Dupire converts an option price or implied-volatility surface into a local-volatility function that depends on stock price and time:

$$dS_t = rS_t dt + \sigma_{\text{loc}}(S_t, t)S_t dW_t.$$

This gives a dynamic stock-price model calibrated to the observed option surface under ideal smooth and arbitrage-free conditions.

Dupire Local-Volatility Method

The Dupire method is implemented in `src/dupire.py`. The project first fits an implied-volatility interpolator over log-moneyness and maturity:

$$x = \log(K/S), \quad \sigma_{\text{imp}} = \sigma_{\text{imp}}(x, T).$$

The code uses linear interpolation where possible and nearest-neighbor fallback where linear interpolation is unavailable. Implied volatilities are clipped to a practical range to avoid extreme numerical behavior.

Next, the code builds a grid of log-moneyness and maturity values, converts log-moneyness back to strikes using

$$K = S_0 e^x,$$

and prices calls on the grid using the Black–Scholes formula with the interpolated implied volatility:

$$C(K, T) = S_0 N(d_1) - K e^{-rT} N(d_2).$$

This produces a numerical call price surface $C(K, T)$.

The local variance is then estimated with the Dupire formula:

$$\sigma_{\text{loc}}^2(K, T) = \frac{\frac{\partial C}{\partial T} + rK \frac{\partial C}{\partial K}}{\frac{1}{2} K^2 \frac{\partial^2 C}{\partial K^2}}.$$

The derivatives are computed numerically using finite differences. When the formula produces invalid or negative local variance, the implementation falls back to implied variance and clips the resulting local volatility to a stable range.

Dupire Delta Proxy

A fully model-consistent Dupire delta would solve the local-volatility pricing problem for the option value $V(S, t)$ and then compute

$$\Delta_{\text{Dupire}} = \frac{\partial V}{\partial S}.$$

For speed and interpretability, this project uses a practical proxy: it looks up the local volatility at the option’s moneyness and maturity, then plugs that volatility into the Black–Scholes call delta formula:

$$\Delta_{\text{proxy}} = N\left(\frac{\log(S/K) + (r + \frac{1}{2}\sigma_{\text{loc}}^2) T}{\sigma_{\text{loc}}\sqrt{T}}\right).$$

This is an educational approximation rather than a production local-volatility delta engine. As a check in `Test/Dupire/Dupire_test.ipynb`, a finite-difference Monte Carlo estimate under local-volatility dynamics produced a numerical delta of approximately 0.47 versus a proxy value of approximately 0.54 for an at-the-money 30-day example. The values are directionally similar, but not identical, which is expected because the proxy is not the full local-volatility PDE delta.

Neural-Network Hedge Method

The neural-network method is implemented in `src/nn_hedge.py` and `src/features.py`. The neural network is a feedforward regression model with three hidden layers:

$$64 \rightarrow 32 \rightarrow 16$$

using ReLU activations and a single scalar output. The output is interpreted as a predicted option hedge ratio.

The model uses the following feature set:

- moneyness and log-moneyness;
- time to maturity T ;
- implied volatility;
- 20-day and 60-day realized stock volatility;
- Black–Scholes call delta;
- option mid price;
- relative bid-ask spread;
- volume and open interest;
- recent stock return and absolute recent stock return.

The Black–Scholes delta is included as one input feature, not as the main target in the historical run. The target is a realized next-period delta estimate:

$$\Delta_{\text{realized}} \approx \frac{\Delta C}{\Delta S} = \frac{C_{t+1} - C_t}{S_{t+1} - S_t},$$

computed within repeated observations of the same strike and expiration. The target is clipped to a practical range of $[-2, 2]$. If repeated contract observations are unavailable, the code warns and falls back to Black–Scholes delta as a proxy target.

The training procedure sorts observations by date, uses the earliest 70% for training, the next 15% for validation, and reserves the remaining period for out-of-sample testing. Features are standardized with `StandardScaler`. Training uses mean squared error loss, Adam optimization, and early stopping with patience 35.

Backtest Design

Both models are evaluated on the same out-of-sample short-call portfolio. The portfolio selection routine chooses a consistently observed call contract and prefers contracts close to at-the-money. The default project configuration is:

- underlying: SPY;
- risk-free rate: $r = 4\%$;
- position: one short call contract;
- contract multiplier: 100;
- rebalancing: daily;
- transaction cost: 1 basis point on stock trades;
- initial cash: 0.

For each day in the out-of-sample portfolio, the strategy converts the option delta into the number of SPY shares needed to offset the short option’s delta:

$$\text{shares} = -(\text{option quantity}) \times \Delta \times \text{contract multiplier}.$$

The backtest tracks cash, stock position, option value, portfolio value, daily P&L, hedging error, transaction costs, and turnover.

The portfolio value is the marked value of the combined hedged position:

$$\text{portfolio value} = \text{short option value} + \text{stock hedge value} + \text{cash}.$$

This should not be interpreted as directly comparable to a long-only SPY buy-and-hold portfolio, because the hedging strategies include a short option liability and dynamic stock rebalancing.

Results Summary

The current run evaluates 63 out-of-sample portfolio observations. The summary metrics saved in `results/strategy_comparison.csv` are:

Model	Total P&L	Sharpe	Mean Abs. Error	Costs
Dupire Delta Hedge	291.24	1.66	34.61	8.17
Neural-Network Delta Hedge	297.33	0.74	74.18	10.90

In this run, the neural-network hedge has slightly higher total P&L, but the Dupire hedge has materially lower P&L volatility, lower mean absolute hedging error, lower RMSE hedging error, lower maximum drawdown, lower transaction costs, and lower turnover. The Dupire strategy therefore appears more stable as a hedge in this particular experiment, while the neural network produces a somewhat higher total P&L at the cost of noisier hedge behavior.

The notebook also includes an optional SPY buy-and-hold market benchmark. This benchmark is useful only as market context. It is not a competing hedging strategy because it does not include a short option liability, does not compute a hedge ratio, and has no option hedging error.

Key Takeaways

- The project demonstrates an end-to-end comparison between a classical financial model and a learning-based hedge ratio model.
- The Dupire method is mathematically grounded in the implied-volatility surface, but it is sensitive to noisy market data and numerical differentiation.
- The neural network can learn hedge ratios from market features and realized option-stock movements, but it requires careful validation and may produce less stable hedge behavior.
- Hedging quality should be judged with risk and error metrics, not only total P&L. Lower hedging error, lower turnover, lower transaction costs, and lower drawdown are central to evaluating hedge effectiveness.
- Better hedging does not necessarily imply a profitable trading strategy. The project studies hedge performance, not an alpha signal or standalone trading strategy.

Limitations and Future Work

Several limitations are important for interpreting the results. The Dupire surface is an educational approximation and uses finite differences on an interpolated surface, which can amplify quote noise. The delta used in the backtest is a Black–Scholes-with-local-volatility proxy rather than a full PDE-derived local-volatility delta. The neural network is relatively simple and is trained on realized next-period delta estimates, which can themselves be noisy. Finally, the backtest uses one selected short-call portfolio and should not be interpreted as a comprehensive live trading evaluation.

Natural extensions include stronger surface smoothing, a full PDE-based Dupire delta calculation, broader option portfolio construction, richer transaction cost modeling, cross-validation across market regimes, and comparison with additional hedge models.